

Recuperatorio programacion avanzada

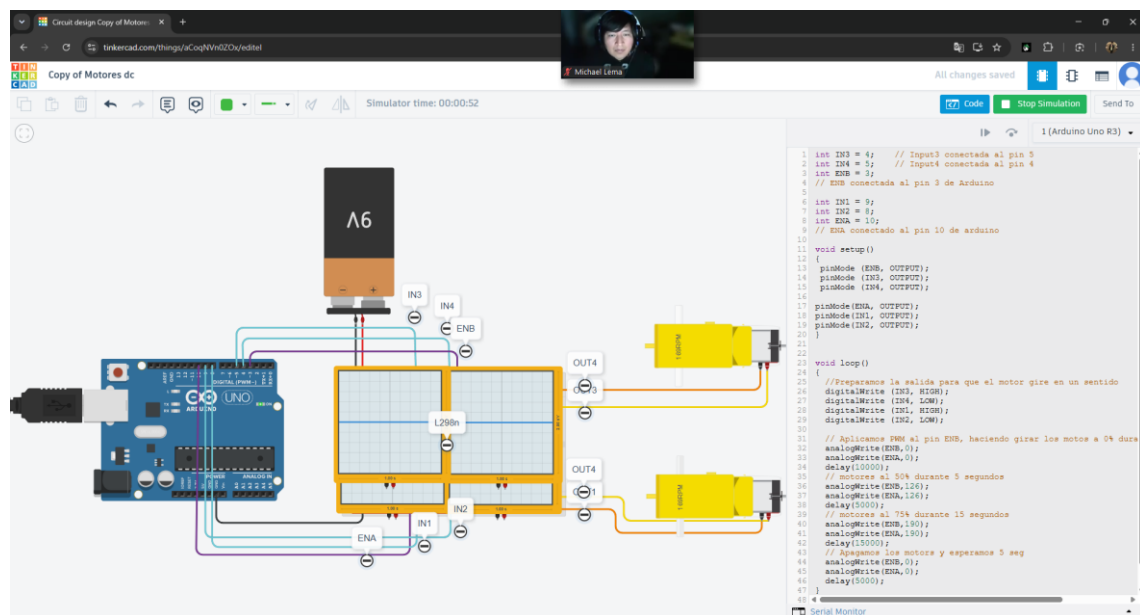
Alumno: Flores, Nicolás

1)

a) Link simulacion:

https://www.tinkercad.com/things/aCoqNVn0ZOx/editel?sharecode=KeX2k2sOcvdaPCbPAxaAf41j1-uSV53_8vCcJaDfFX0

c)



Codigo:

int IN3 = 4; // Input3 conectada al pin 5

int IN4 = 5; // Input4 conectada al pin 4

int ENB = 3;

// ENB conectada al pin 3 de Arduino

int IN1 = 9;

int IN2 = 8;

int ENA = 10;

// ENA conectado al pin 10 de arduino

```
void setup()
```

```
{
```

```
  pinMode (ENB, OUTPUT);
```

```
  pinMode (IN3, OUTPUT);
```

```
  pinMode (IN4, OUTPUT);
```

```
  pinMode(ENA, OUTPUT);
```

```
  pinMode(IN1, OUTPUT);
```

```
  pinMode(IN2, OUTPUT);
```

```
}
```

```
void loop()
```

```
{
```

```
  //Preparamos la salida para que el motor gire en un sentido
```

```
  digitalWrite (IN3, HIGH);
```

```
  digitalWrite (IN4, LOW);
```

```
  digitalWrite (IN1, HIGH);
```

```
  digitalWrite (IN2, LOW);
```

```
  // Aplicamos PWM al pin ENB, haciendo girar los motos a 0% durante 10 segundos
```

```
  analogWrite(ENB,0);
```

```
  analogWrite(ENA,0);
```

```
  delay(10000);
```

```
  // motores al 50% durante 5 segundos
```

```
  analogWrite(ENB,126);
```

```
  analogWrite(ENA,126);
```

```
  delay(5000);
```

```

// motores al 75% durante 15 segundos

analogWrite(ENB,190);

analogWrite(ENA,190);

delay(15000);

// Apagamos los motors y esperamos 5 seg

analogWrite(ENB,0);

analogWrite(ENA,0);

delay(5000);

}

```

b) Este código fue modificado para que los motores comiencen al 0% ósea apagados por 10 segundos. Lo que se hace es colocar en el programa un delay (10000) y (ENB , ENA, 0).

El siguiente paso es que pasado los 10 segundos los motores se activen y giren al 50% durante 5 segundos. Para ellos colocamos delay (5000) y (ENB, ENA 126) donde, 126 aproximadamente correspondería a un 50%.

Por ultimo luego de los 5 segundos los moteres giran a un 70% durante 15 segundos, donde: delay (15000) y (ENB, ENA, 190) donde 190 serian aproximadamente el 75%,

3)

Protocolos de comunicación:

Los protocolos de comunicación permiten qu Arduino intercambie información con otros dispositivos, como los sensores, modulos de comunicacio, sensores y otros microcontroladores.

Los que vimos en clases son la comuicacion serial (UART) y la comunicación I2C.

En el caso de la comunicación serial consiste en transmitirt información de manera asincrona, enviando los datos un bit a la vez a través de una línea de comunicación.

En comunicación tiene 2 lines principales:

TX (TRANSMISION): por donde se envían los datos.

RX(RECEPCION): por donde se reciben los datos.

Ademas los dispositivos deben compartir GND.

No utiliza una línea de reloj para sincronizarse.

Un ejemplo es la conexión de la placa Arduino y la computadora a través del cable usb para enviar datos al monitor serial.

Las ventajas es que es compatible con dispositivos como modulos bluetooth, wifi y gps.

Requiere pocos cables y es mas sencillo de comprender.

La comunicación I2C en cambio es un bus de comunicación síncrono que utiliza 2 cables.

SDA (LINEA DE DATOS): transporta los datos.

SCL (LINEA DE RELOJ): transporta la señal del reloj que sincroniza la comunicación.

Permite conectar varios dispositivos en paralelo compartiendo los mismos cables usando direcciones de hardware único de 7 a 10 bits.

Maestro: Es el dispositivo que comunica la línea del reloj, inicia y finaliza la comunicación y decide a que dispositivo envia y pide datos.

Esclavo: Es el dispositivo que tiene una dirección única en el hardware. No puede iniciar la comunicación , solo responde cuando el maestro se comunica por su dirección.

Ejemplo: el maestro seria el Arduino que envia datos de texto a la pantalla lcd la cual es el esclavo.

Las principales ventajas de esta comunicación es que solo usa 2 cables para soportar muchos dispositivos.

3)

Código:

```
#include <LiquidCrystal.h>
```

```
// Inicialización de la librería con los pines de la interfaz
```

```
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);
```

```
// Pin del sensor de temperatura
```

```
const int pinTemperatura = A0;
```

```
// Variable global para almacenar el valor procesado
```

```
float temperatura = 0.0;
```

```
void setup() {
```

```
    // Configuración de la LCD (16 columnas y 2 filas)
```

```
    lcd.begin(16, 2);
```

```
    // Pantalla de inicio
```

```
    lcd.setCursor(0, 0);
```

```
    lcd.print("  NF GARAGE  ");
```

```
    lcd.setCursor(0, 1);
```

```
    lcd.print(" TERMOMETRO  ");
```

```
    delay(2000);
```

```
    lcd.clear();
```

```
}
```

```
void loop() {
```

```
    // 1. Leer y calcular la temperatura
```

```
    leerTemperatura();
```

```
    // 2. Mostrar el resultado en la LCD
```

```
    mostrarPantalla();
```

```
    // Espera de 1 segundo antes de la siguiente lectura
```

```
    delay(1000);
```

```
}
```

```
// Módulo para leer el pin analógico y convertirlo a grados Celsius
```

```

void leerTemperatura() {

    int lectura = analogRead(pinTemperatura);

    // Conversión del valor analógico (0-1023) a voltaje (0-5V)
    float voltaje = lectura * (5.0 / 1023.0);

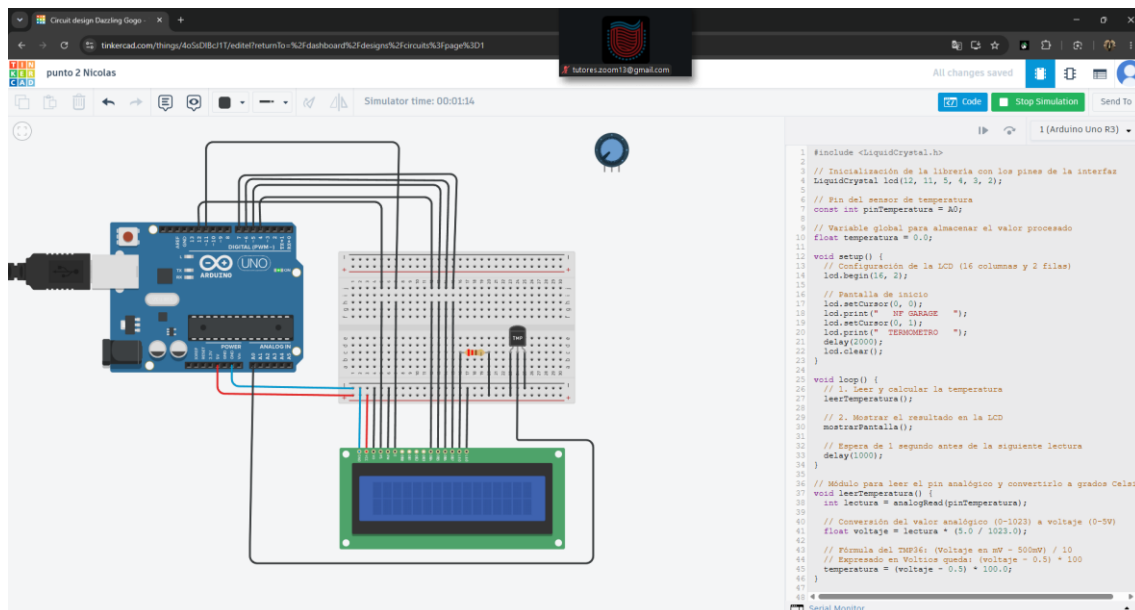
    // Fórmula del TMP36: (Voltaje en mV - 500mV) / 10
    // Expresado en Voltios queda: (voltaje - 0.5) * 100
    temperatura = (voltaje - 0.5) * 100.0;
}

// Módulo para el manejo de la pantalla LCD
void mostrarPantalla() {

    // Centramos el texto en la primera fila
    lcd.setCursor(0, 0);
    lcd.print(" TEMPERATURA ");

    // Mostramos el valor en la segunda fila
    lcd.setCursor(4, 1);
    lcd.print(temperatura, 1); // Muestra el valor con 1 decimal
    lcd.print(" C ");      // Espacios al final por prolijidad
}

```



Link:

https://www.tinkercad.com/things/4oSsDlBcJ1T/editel?returnTo=%2Fdashboard%2Fdesigns%2Fcircuits%3Fpage%3D1&sharecode=ng0YtJsOXUv_CMLEKQi-IBKt_wyVt0IHg2bTYsv4VGw

Profe no llegue con el tiempo, no me estaría funcionando, no llego a verificar donde me estoy equivocando, pero quería usar un sensor de temperatura para que lcd me muestre.